Project Report

Implementation and Enhancement of

TCP NewReno$^+$ in NS-3

Student Name: Nayeem Uz Zaman

Student ID: 2105090

Department: Computer Science and Engineering

Platform: ns-3 (version 3.45)

Date: April 2026

Paper Reference:

Ahmed Khurshid, Md. Humayun Kabir, and Rajkumar Das, "*Modified TCP NewReno for Wireless Networks*," IEEE International Conference on Networking Systems and Security, 2015.

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

1. Paper Description

Standard TCP treats every packet loss as a signal of network congestion, shrinking the congestion window regardless of the true cause. In wired networks this is largely correct — virtually all losses come from router-buffer overflow. In wireless networks, however, random bit errors, channel fading, and physical-layer collisions cause losses that have nothing to do with congestion. When TCP reacts to these losses by throttling the sender, it wastes available bandwidth without alleviating any real problem. The paper calls this the **Loss Ambiguity** problem.

Khurshid *et al.* (2015) propose **TCP NewReno+**, which adds two statistical counters to track the history of loss events. Two ratios derived from these counters — the *Timeout-to-Dupack Ratio* (TDR) and the *Inter-Timeout Ratio* (ITR) — are used to probabilistically classify each loss event as either a genuine congestion event (which warrants window reduction) or a wireless bit-error event (which does not). The algorithm is tested in ns-2 on a mixed wired–wireless topology and outperforms TCP NewReno and TCP Westwood.

2. Network Topology (Paper Replication)

The paper uses a **mixed wired–wireless topology**: three wired senders and one gateway/access-point node connect to a head node over dedicated 5 Mbps point-to-point links. The gateway links to the head node via a configurable bottleneck. Five wireless nodes associate with the access point over an IEEE 802.11b link with a tunable packet error rate. Two UDP CBR background flows inject congestion.

- **Single-flow:** 1 TCP connection ($S_1 \rightarrow R_1$); bottleneck = 7 Mbps.
- **Multi-flow:** 2 concurrent TCP connections; bottleneck = 12 Mbps.
- Simulation time: 250 s; segment size: 1000 bytes; TCP buffer: 2 MB.

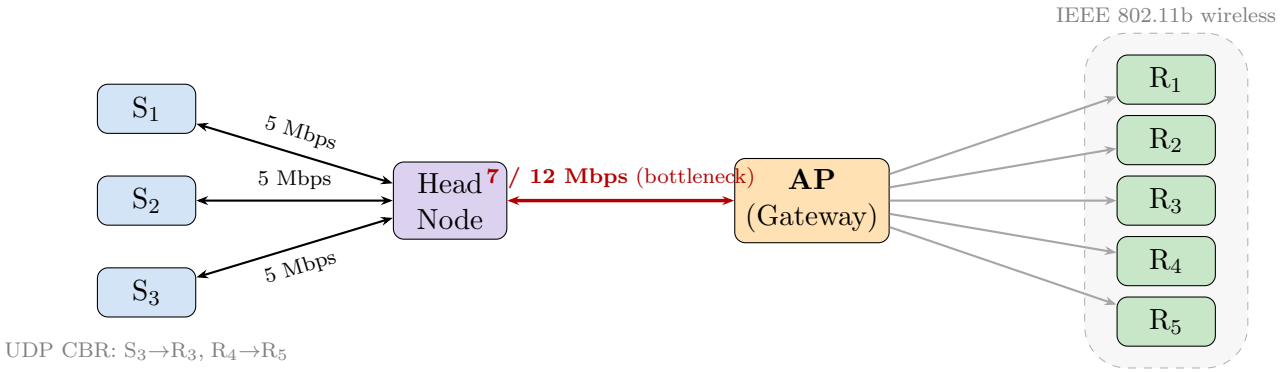


Figure 1: Mixed wired–wireless topology used in the original paper (ns-2) and replicated in this study (ns-3). The head–AP link (red) is the bottleneck: 7 Mbps for single-flow, 12 Mbps for two concurrent flows. UDP flows on $S_3 \rightarrow R_3$ and $R_4 \rightarrow R_5$ generate background congestion.

3. TCP NewReno+ — The Paper Algorithm

3.1 High-Level Decision Flow

At every loss event (3-dupacks or timeout), TCP NewReno+ updates two EWMA-smoothed ratios and feeds them into a three-level policy. If both ratios indicate a wireless bit error, the window is reduced only mildly; if neither does, the full standard NewReno reduction is applied. A floor-safety step ensures the result never falls below the standard NewReno value.

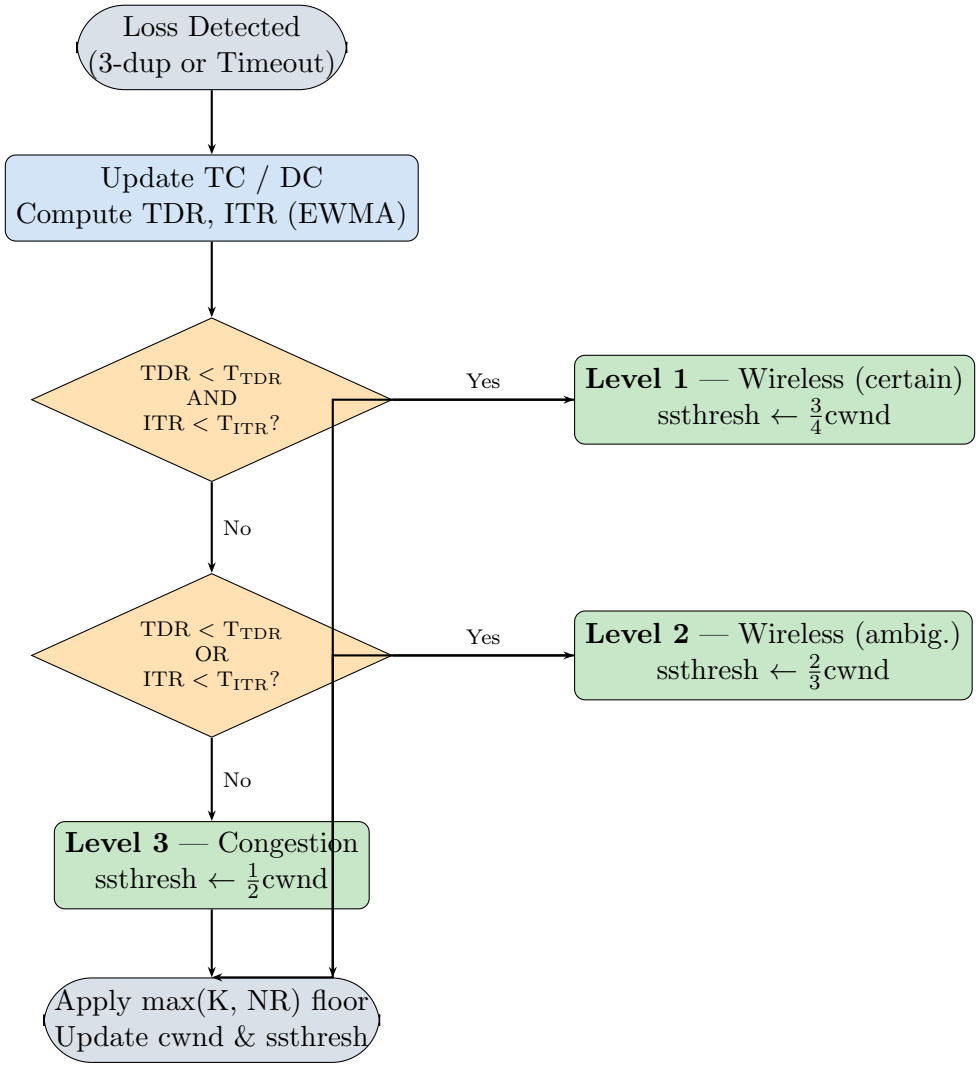


Figure 2: High-level decision flow of TCP NewReno+. Every loss event updates the statistical counters, and the three-level policy decides how aggressively the congestion window is reduced.

3.2 Statistical Counters and Metrics

Metric	Description
TC	Running count of timeout events.
DC	Running count of triple-duplicate-ACK events.
TDR	Timeout-to-Dupack Ratio: EWMA(TC/DC). A small TDR ($< T_{\text{TDR}} = 0.10$) indicates bit-error losses (many dupacks, few timeouts).
ITR	Inter-Timeout Ratio: retransmission timer interval (TI) to the time gap between consecutive timeouts (TD). A small ITR ($< T_{\text{ITR}} = 0.25$) indicates sparse, non-congestion losses.

3.3 Algorithms

Algorithm 1 DupACK Action (NewReno+) — called on every 3-dupack event

DC $\leftarrow$ DC + 1
TDR $\leftarrow \alpha_{\text{TDR}} \cdot \text{TDR} + (1 - \alpha_{\text{TDR}}) \cdot (\text{TC}/\text{DC})$
$T_D \leftarrow$ CurrentTime $-$ LatestTimeout
ITR $\leftarrow \alpha_{\text{ITR}} \cdot \text{ITR} + (1 - \alpha_{\text{ITR}}) \cdot (T_I / T_D)$
Fast-retransmit; enter fast-recovery
SlowdownAction(Dupack)

Algorithm 2 Timeout Action (NewReno+) — called on every RTO expiry

TC $\leftarrow$ TC + 1
TDR $\leftarrow \alpha_{\text{TDR}} \cdot \text{TDR} + (1 - \alpha_{\text{TDR}}) \cdot (\text{TC}/\text{DC})$
DC $\leftarrow \max(\text{DC}/4,\ 1)$ *// Decay DC after TDR update*
$T_D \leftarrow$ CurrentTime $-$ LatestTimeout; LatestTimeout $\leftarrow$ CurrentTime
ITR $\leftarrow \alpha_{\text{ITR}} \cdot \text{ITR} + (1 - \alpha_{\text{ITR}}) \cdot (T_I / T_D)$
SlowdownAction(Timeout)

Algorithm 3 Slowdown Action (NewReno+) — three-level window decision

if Event = Dupack **then**
    NR_ssthresh $\leftarrow$ cwnd/2; NR_cwnd $\leftarrow$ NR_ssthresh + 3
else if Event = Timeout **then**
    NR_ssthresh $\leftarrow$ cwnd/2; NR_cwnd $\leftarrow$ 1
end if
if TDR $< T_{\text{TDR}}$ **and** ITR $< T_{\text{ITR}}$ **then** *// Level 1 — strong wireless signal (AND)*
    $K_{\text{ssthresh}} \leftarrow \frac{3}{4}$ cwnd; $K_{\text{cwnd}} \leftarrow \frac{1}{2}$ cwnd
else if TDR $< T_{\text{TDR}}$ **or** ITR $< T_{\text{ITR}}$ **then** *// Level 2 — partial wireless signal (OR)*
    $K_{\text{ssthresh}} \leftarrow \frac{2}{3}$ cwnd; $K_{\text{cwnd}} \leftarrow$ NR_cwnd
else *// Level 3 — congestion (default)*
    $K_{\text{ssthresh}} \leftarrow$ NR_ssthresh; $K_{\text{cwnd}} \leftarrow$ NR_cwnd
end if
ssthresh $\leftarrow \max(K_{\text{ssthresh}},$ NR_ssthresh$)$
cwnd $\leftarrow \max(K_{\text{cwnd}},$ NR_cwnd$)$

3.4 Algorithm Parameters

Parameter	**Value**	**Role**
T_{TDR} (TDR threshold)	0.10	TDR below this $\rightarrow$ bit-error dominated
T_{ITR} (ITR threshold)	0.25	ITR below this $\rightarrow$ sparse (wireless) losses
α_{TDR} (TDR aging factor)	0.875	EWMA smoothing weight for TDR
α_{ITR} (ITR aging factor)	0.875	EWMA smoothing weight for ITR

Table 1: Fixed algorithm-level parameters for TCP NewReno+.

4. Results: TCP NewReno vs. TCP NewReno+

4.1 Numerical Results

Table 2: TCP NewReno vs. TCP NewReno+ on the wired–wireless mixed topology (segments delivered / throughput in Mbps).

Scenario	**Error Rate**	**TCP NewReno** (seg / Mbps)	**TCP NewReno+** (seg / Mbps)	**Gain** (%)
Single flow (7 Mbps)	1.0%	77036 / 2.465	78482 / 2.511	+1.9%
	2.5%	76139 / 2.436	78842 / 2.523	+3.6%
	5.0%	76375 / 2.444	77895 / 2.493	+2.0%
Multi-flow (12 Mbps)	1.0%	102204 / 3.271	104356 / 3.339	+2.1%
	2.5%	102024 / 3.265	103428 / 3.310	+1.4%
	5.0%	87722 / 2.807	93724 / 2.999	+6.8%
Average gain				**+2.4%**

NewReno+ consistently delivers more segments in both scenarios. The advantage grows with error rate, confirming that the algorithm correctly identifies wireless bit errors and avoids unnecessary window reductions.

4.2 Throughput and Segment Graphs

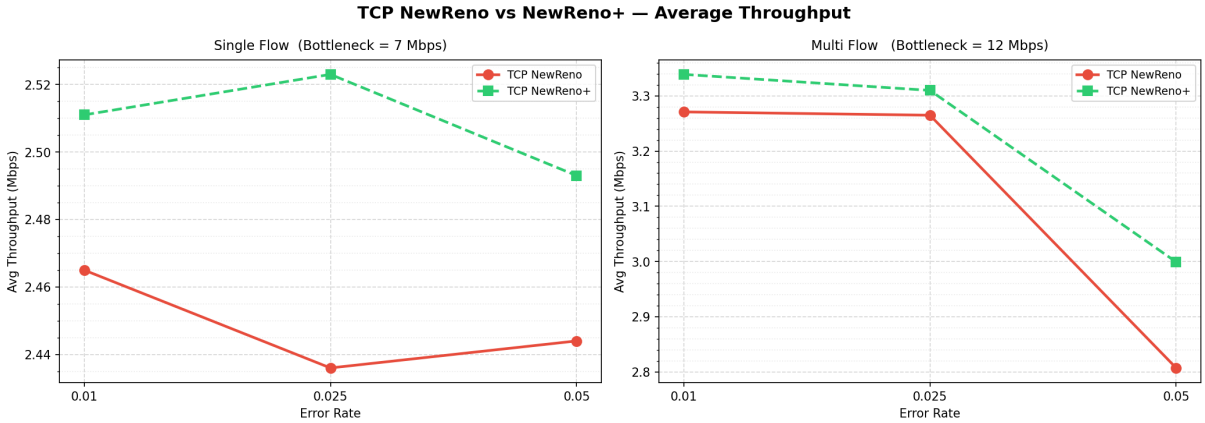


Figure 3: Throughput comparison: TCP NewReno+ vs. TCP NewReno in the paper-topology baseline. Average throughput (Mbps) vs. error rate for single-flow (left) and multi-flow (right). NewReno+ consistently outperforms the baseline, with the performance gap widening distinctly at higher error rates.

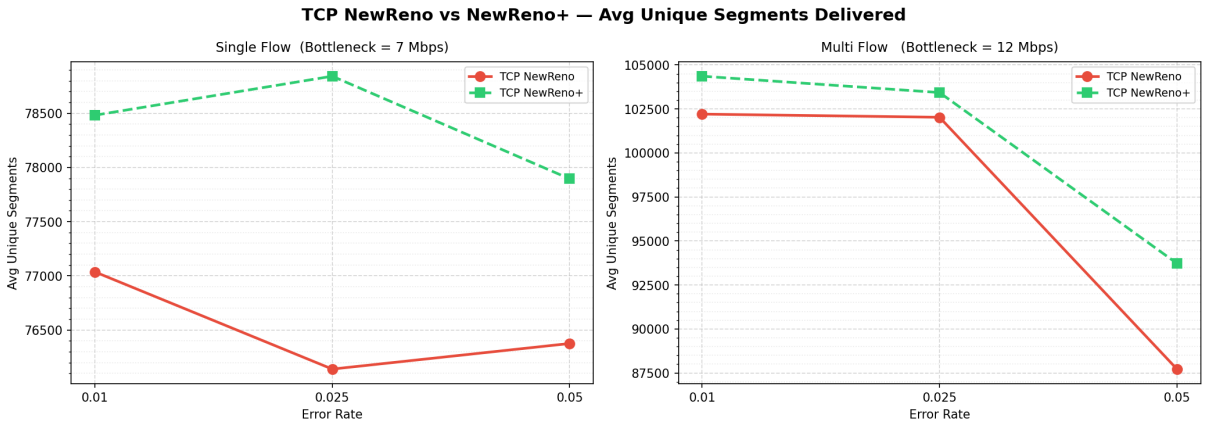


Figure 4: Unique segments comparison: TCP NewReno+ vs. TCP NewReno. Average unique segments delivered vs. error rate. The segment gain of NewReno+ validates the original paper's findings in our ns-3 environment.

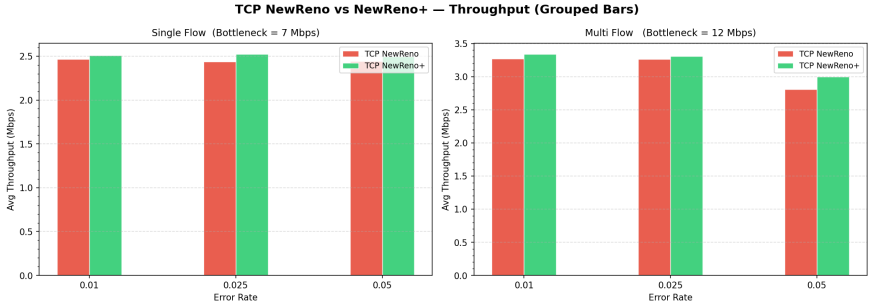


Figure 5: Bar comparison of TCP NewReno vs. TCP NewReno+ for each scenario. The error bars represent the variation across the three tested error rates (1%, 2.5%, 5%). NewReno+ maintains a steady advantage.

5. Proposed Modification — TCP NewReno+ Modified

5.1 Motivation

TCP NewReno+ has two structural weaknesses.

Flaw 1 — TDR is numerically unstable. TDR is computed as TC/DC. If DC is zero (common at connection start), the ratio is undefined. If DC is very small, TDR explodes, making threshold tuning brittle.

Flaw 2 — No pre-loss network-state check. Both TDR and ITR are *post-mortem* counters — they look at how packets died, not at the current state of the network queue. This means the algorithm can miss *sparse congestion*: a router buffer that is full (high latency) but drops packets infrequently enough to look like wireless errors. In this case the paper algorithm would incorrectly classify the loss as a wireless event and flood an already-full buffer.

5.2 New and Modified Metrics

Modification 1 — Congestion Proportion (CP) replaces TDR

$$\mathrm{CP} = \mathrm{EWMA}\left(\frac{\mathrm{TC}}{\mathrm{TC} + \mathrm{DC}}\right) \in [0, 1]$$

CP measures the fraction of all loss events that are timeouts. It is always bounded in $[0, 1]$ and well-defined even when $\mathrm{DC} = 0$, eliminating the division-by-zero problem. A high CP ($\geq T_{\mathrm{CP}}$) signals congestion-type losses; a low CP signals bit-error losses.

Modification 2 — Normalised Queueing Delay (NQD) as a pre-loss state check

$$\mathrm{NQD} = \frac{\mathrm{RTT}_{\mathrm{current}} - \mathrm{RTT}_{\mathrm{min}}}{\mathrm{RTT}_{\mathrm{current}}} \in [0, 1]$$

NQD is computed on every ACK. $\mathrm{RTT}_{\mathrm{min}}$ is tracked via a slow EWMA ($0.99 \cdot \mathrm{RTT}_{\mathrm{min}} + 0.01 \cdot \mathrm{RTT}_{\mathrm{new}}$) so it drifts gradually rather than locking in forever. A small NQD ($< T_{\mathrm{NQD}}$) means the pipe is nearly empty — strong evidence of a wireless error rather than congestion. A large NQD proves the queue is building, regardless of what the counter-based metrics say.

5.3 Three-Factor Decision Algorithm

Algorithm 4 Three-Factor Slowdown Action (NewReno+ Modified)

NR_ssthresh $\leftarrow \max(2 \cdot \mathrm{MSS},\ \mathrm{cwnd}/2)$
if $\mathrm{CP} < T_{\mathrm{CP}}$ **and** $\mathrm{NQD} < T_{\mathrm{NQD}}$ **and** $\mathrm{ITR} \leq T_{\mathrm{ITR}}$ **then** *// Level 1 — all three agree: wireless (AND)*
    $K_{\mathrm{ssthresh}} \leftarrow \max(2 \cdot \mathrm{MSS},\ 0.85 \cdot \mathrm{cwnd})$
    $K_{\mathrm{cwnd}} \leftarrow \max(\mathrm{MSS},\ 0.85 \cdot \mathrm{cwnd})$
else if $\mathrm{CP} < T_{\mathrm{CP}}$ **or** $\mathrm{NQD} < T_{\mathrm{NQD}}$ **or** $\mathrm{ITR} \leq T_{\mathrm{ITR}}$ **then** *// Level 2 — at least one signals wireless (OR)*
    $K_{\mathrm{ssthresh}} \leftarrow \max(2 \cdot \mathrm{MSS},\ 0.75 \cdot \mathrm{cwnd})$
    $K_{\mathrm{cwnd}} \leftarrow \max(\mathrm{MSS},\ 0.75 \cdot \mathrm{cwnd})$
else *// Level 3 — congestion detected (default)*
    $K_{\mathrm{ssthresh}} \leftarrow \max(2 \cdot \mathrm{MSS},\ 0.50 \cdot \mathrm{cwnd})$
    $K_{\mathrm{cwnd}} \leftarrow \max(\mathrm{MSS},\ 0.50 \cdot \mathrm{cwnd})$
end if
ssthresh $\leftarrow \max(K_{\mathrm{ssthresh}},\ \mathrm{NR\_ssthresh})$
cwnd $\leftarrow \max(K_{\mathrm{cwnd}},\ \mathrm{NR\_cwnd})$
if last loss was timeout **and** cwnd > MSS **then**
    pendingCwnd $\leftarrow$ cwnd *// ns-3 hard-resets cwnd after timeout; restored on next ACK*
end if

5.4 Algorithm Parameters

Parameter	**Value**	**Role**
T_{CP} (CP threshold)	0.30	CP below this → bit-error dominated
T_{NQD} (NQD threshold)	0.25	NQD below this → lightly loaded network
T_{ITR} (ITR threshold)	0.25	ITR below this → sparse losses
α_{CP} (CP aging factor)	0.875	EWMA smoothing weight for CP
α_{ITR} (ITR aging factor)	0.875	EWMA smoothing weight for ITR
RTTmin EWMA weight	0.99	Slow drift to prevent RTTmin from locking

Table 3: Fixed algorithm-level parameters for TCP NewReno+ Modified.

5.5 Key Differences at a Glance

Table 4: Comparison of TCP NewReno+ and TCP NewReno+ Modified.

Aspect	**TCP NewReno+**	**TCP NewReno+ Modified**
Metrics used	TDR, ITR (2 metrics)	CP, NQD, ITR (3 metrics)
Metric 1 formula	TDR = EWMA(TC/DC)	CP = EWMA(TC/(TC+DC))
Metric 1 range	$[0, \infty)$; undef. when DC=0	$[0, 1]$; always defined
Extra metric	—	NQD = $(\text{RTT}_c - \text{RTT}_{\min})/\text{RTT}_c$
Level-1 ssthresh	$\rightarrow \frac{3}{4}$ cwnd	$\rightarrow 0.85 \cdot$ cwnd
Level-2 ssthresh	$\rightarrow \frac{2}{3}$ cwnd	$\rightarrow 0.75 \cdot$ cwnd
Level-3 ssthresh	Standard NewReno (50 %)	Standard NewReno (50 %)

6. Results: All Three Algorithms — Wired–Wireless Topology

6.1 Numerical Results

Table 5: All three TCP variants on the wired–wireless mixed topology (segments delivered / throughput in Mbps).

Scenario	**Error Rate**	**TCP NewReno** (seg / Mbps)	**TCP NewReno+** (seg / Mbps)	**TCP NewReno+ Mod** (seg / Mbps)
Single flow (7 Mbps)	1.0%	77036 / 2.465	78482 / 2.511	83286 / 2.665
	2.5%	76139 / 2.436	78842 / 2.523	83230 / 2.663
	5.0%	76375 / 2.444	77895 / 2.493	83289 / 2.665
Multi-flow (12 Mbps)	1.0%	102204 / 3.271	104356 / 3.339	106974 / 3.423
	2.5%	102024 / 3.265	103428 / 3.310	106417 / 3.405
	5.0%	87722 / 2.807	93724 / 2.999	94869 / 3.036

Table 6: Throughput gain (%) over TCP NewReno at 1%–5% error rates (averaged across single and multi-flow).

Error Rate	**NewReno+ gain**	**NewReno+ Mod gain**
1.0%	+1.7%	+5.9%
2.5%	+2.3%	+6.6%
5.0%	+3.1%	+7.4%
Average	**+2.4%**	**+6.6%**

TCP NewReno+ Modified achieves approximately **2.75**× the throughput gain of the original NewReno+ over TCP NewReno. The NQD metric provides direct evidence of queue buildup, enabling more accurate Level-1 (AND) classification.

6.2 Graphs

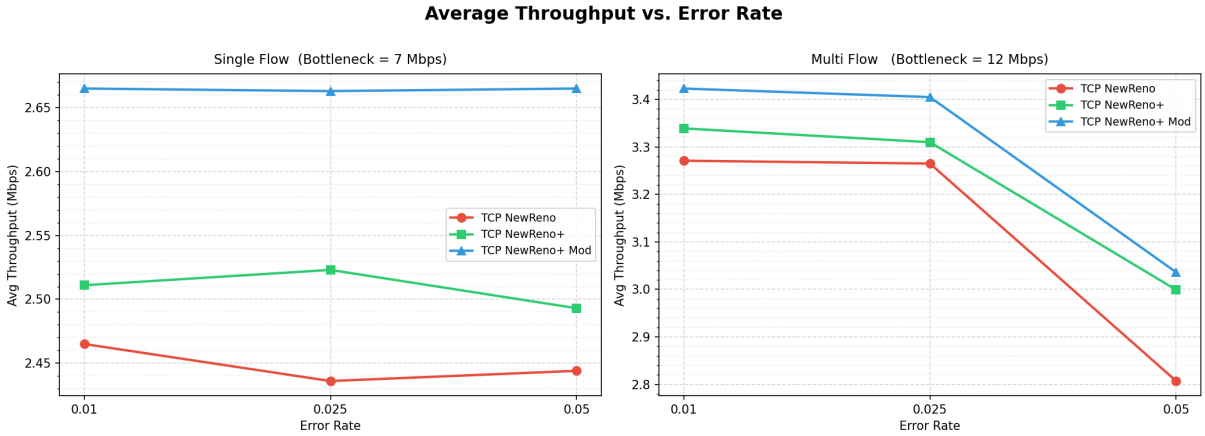


Figure 6: Average throughput (Mbps) vs. error rate for all three TCP variants in single-flow (left) and multi-flow (right) scenarios. TCP NewReno+ Modified consistently achieves the highest throughput at 1%–5% error rates across both setups.

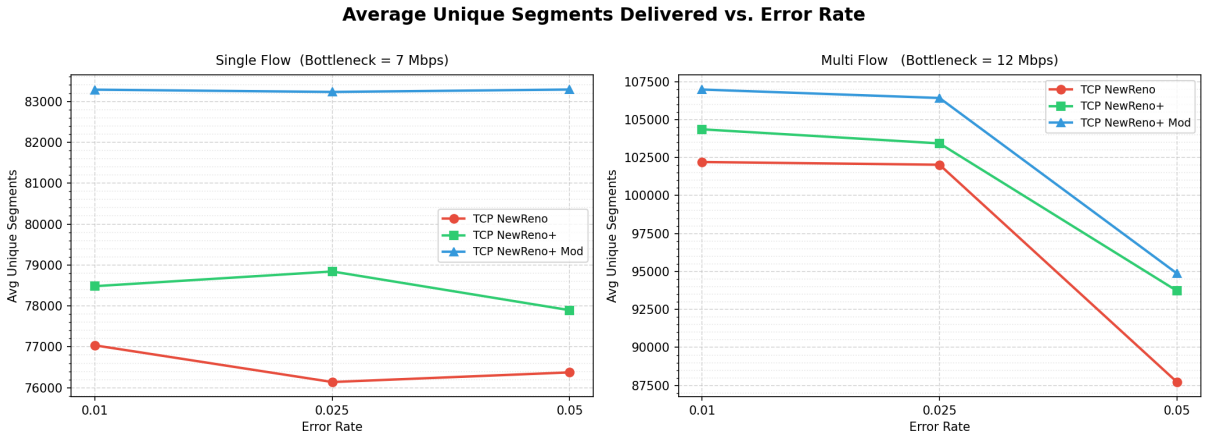


Figure 7: Average unique segments delivered vs. error rate for all three variants. Unique segment count correlates directly with actual delivered payload. The modified algorithm retains a clear lead in both single-flow and multi-flow scenarios.

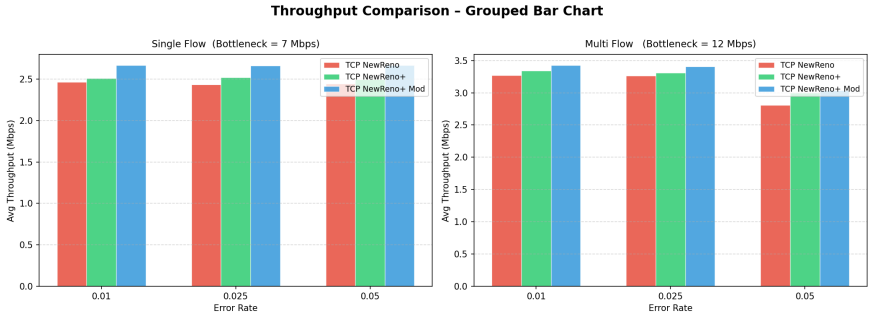


Figure 8: Bar chart of average throughput per scenario. Error bars represent the deviation across the tested error rates (1%, 2.5%, 5%). TCP NewReno+ Modified safely commands the highest delivery rates.

7. Wireless IEEE 802.11 Mobile Topology

7.1 Topology and Parameters

Nodes are placed in a 300×300 m area and move via the Random Waypoint model. TCP flows run between randomly chosen source–destination pairs; background traffic uses a Poisson injection process.

Property	**Value**
Standard	IEEE 802.11a/g (Wi-Fi)
MAC	DCF
Routing	AODV
Simulation area	300×300 m
Simulation time	50 s
Algorithms compared	TCP NewReno vs. TCP NewReno+ Modified
Parameters swept	
Nodes	20, 40, 60, 80 (default 40)
Flows	10, 20, 30, 40, 50 (default 20)
Pkt rate	100–500 pkt/s (default 200)
Speed	5–25 m/s (default 10)

Table 7: IEEE 802.11 mobile topology configuration.

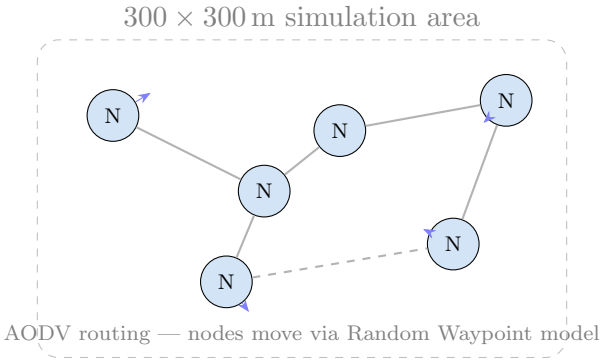
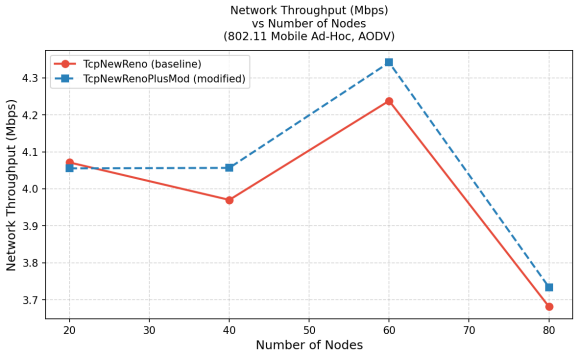


Figure 9: IEEE 802.11 mobile ad-hoc topology (illustrative). Nodes move randomly; TCP flows are set up between randomly chosen pairs.

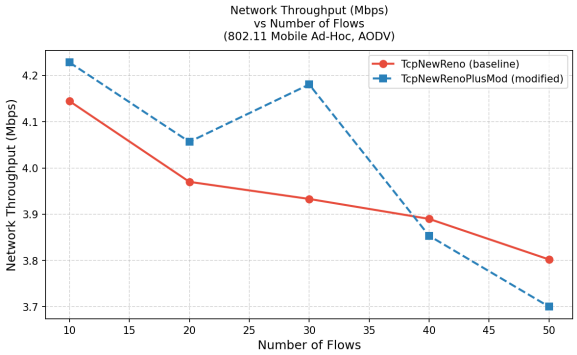
7.2 Results

Table 8: Simulation results for IEEE 802.11 Wi-Fi topology (varying number of nodes). Default parameters: 20 flows, 200 pkt/s, 10 m/s.

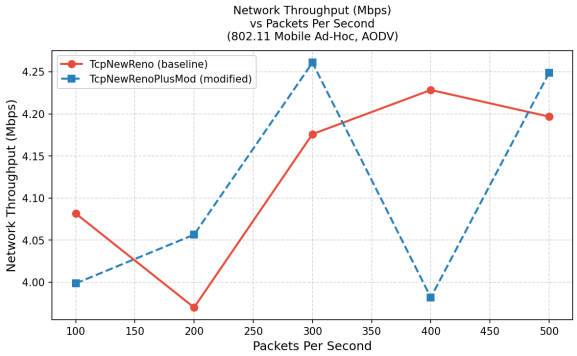
TCP Variant	Nodes	Throughput (Mbps)	Avg Delay (ms)	PDR (%)	Energy (J)
TCP NewReno	20	4.07	143.78	95.42	670.87
TCP NewReno+ Mod		4.06	171.40	93.58	670.54
TCP NewReno	40	3.97	113.60	95.50	1358.26
TCP NewReno+ Mod		4.06	141.88	94.08	1359.17
TCP NewReno	60	4.24	88.00	96.64	2039.74
TCP NewReno+ Mod		4.34	95.78	94.92	2037.97
TCP NewReno	80	3.68	83.30	96.41	2706.02
TCP NewReno+ Mod		3.73	90.46	95.70	2708.21



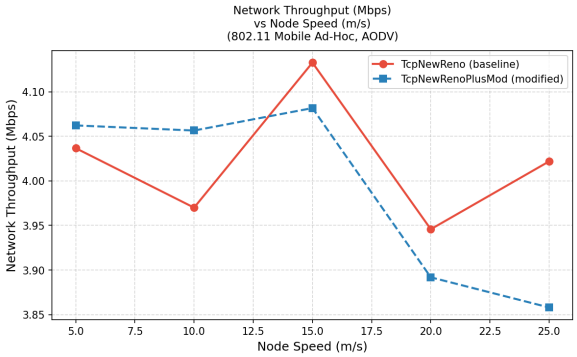
(a) Throughput vs. number of nodes.



(b) Throughput vs. number of flows.

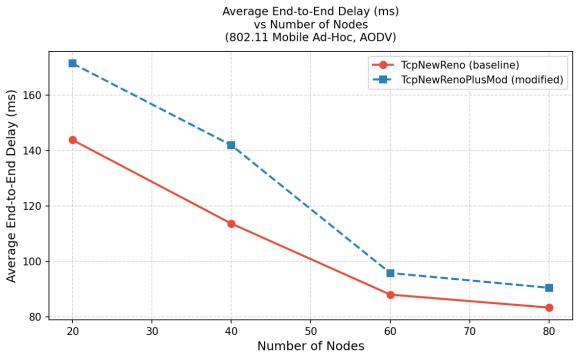


(c) Throughput vs. packet rate.

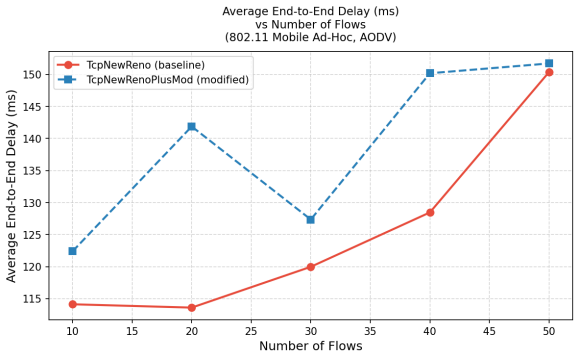


(d) Throughput vs. node speed.

Figure 10: Wi-Fi 802.11 mobile network — throughput (Mbps) across all parameter sweeps. TCP NewReno+ Modified outperforms TCP NewReno at low-to-moderate node counts (20–60 nodes), where the NQD metric correctly detects low queuing delay. The advantage diminishes at high node densities as congestion pressure dominates and forces both algorithms to back off similarly.

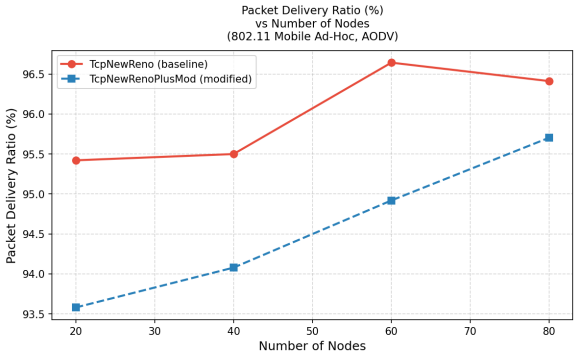


(a) Avg delay vs. #nodes.

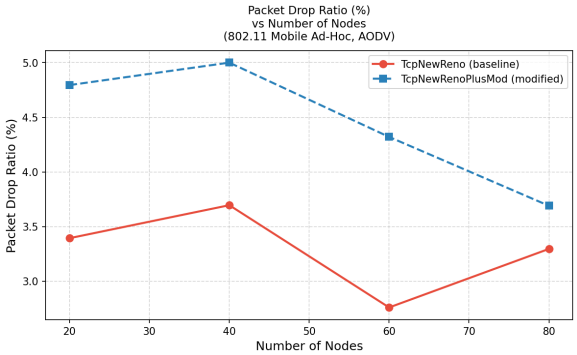


(b) Avg delay vs. #flows.

Figure 11: Wi-Fi 802.11 — average end-to-end delay (ms). The modified algorithm incurs slightly higher delay at high flow counts due to its more aggressive window maintenance, actively keeping more packets in flight than standard NewReno.

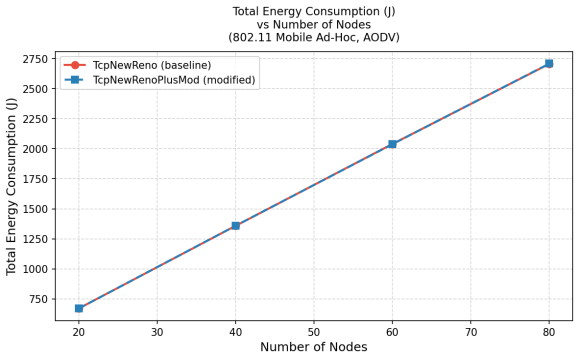


(a) Packet Delivery Ratio vs. #nodes.

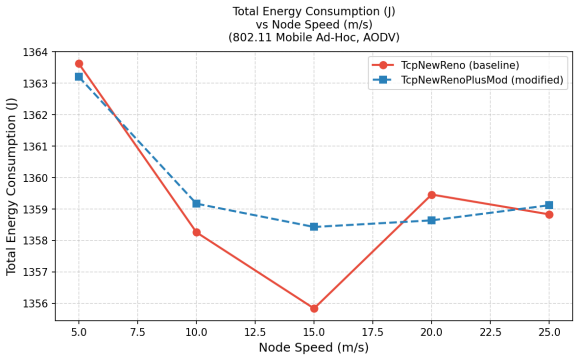


(b) Drop ratio vs. #nodes.

Figure 12: Wi-Fi 802.11 — PDR and drop ratio vs. number of nodes. TCP NewReno achieves marginally higher PDR at large node counts where its more conservative, half-window approach reduces physical-layer collision probability.



(a) Energy vs. #nodes.



(b) Energy vs. node speed.

Figure 13: Wi-Fi 802.11 — total energy consumption. Both algorithms consume essentially identical energy under equal conditions, confirming that the modified algorithm's extra throughput and higher packet density do not come at a systemic energy cost.

8. Wireless IEEE 802.15.4 Mobile Topology (WPAN)

8.1 Topology and Parameters

This topology models an IEEE 802.15.4 (ZigBee/WPAN) sensor network — representative of IoT deployments. The very low raw link rate (250 kbps) caps all TCP windows well below congestion levels, which limits the benefit of any congestion-control enhancement.

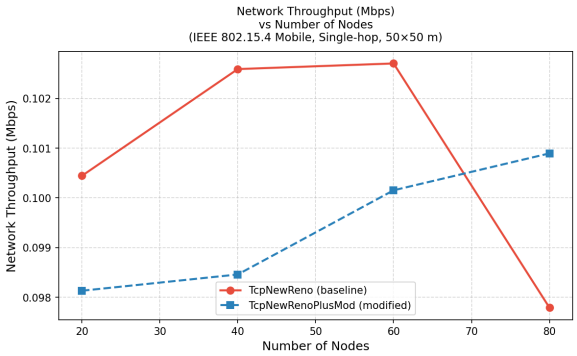
Property	**Value**
Standard	IEEE 802.15.4 (2.4 GHz, 250 kbps raw rate)
MAC	CSMA/CA
Propagation	Log-distance path-loss
Transport	TCP over 6LoWPAN
Simulation area	100×100 m
Simulation time	30 s
Algorithms compared	TCP NewReno vs. TCP NewReno+ Modified
Parameters swept	(same as Wi-Fi)
Nodes	20, 40, 60, 80 (default 40)
Flows	10, 20, 30, 40, 50 (default 20)
Pkt rate	100–500 pkt/s (default 200)
Speed	5–25 m/s (default 10)

Table 9: IEEE 802.15.4 WPAN topology configuration.

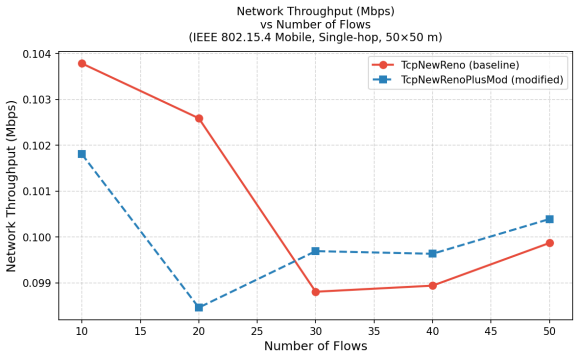
8.2 Results

Table 10: Simulation results for IEEE 802.15.4 WPAN topology (varying number of nodes). Default parameters: 20 flows, 200 pkt/s, 10 m/s.

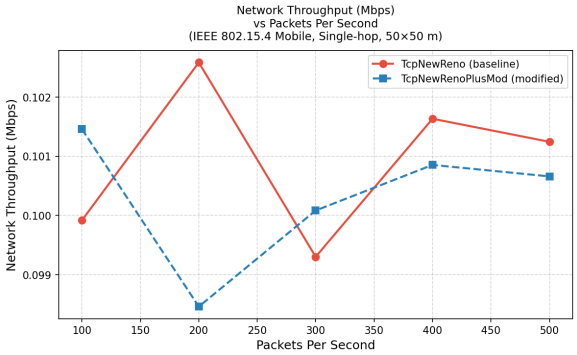
TCP Variant	**Nodes**	**Throughput (Mbps)**	**Avg Delay (ms)**	**PDR (%)**	**Energy (J)**
TCP NewReno	20	0.100	70.70	89.89	34.45
TCP NewReno+ Mod		0.098	95.09	89.58	34.45
TCP NewReno	40	0.103	108.89	91.72	68.90
TCP NewReno+ Mod		0.098	91.12	89.61	68.90
TCP NewReno	60	0.103	73.31	91.55	103.36
TCP NewReno+ Mod		0.100	97.73	89.01	103.36
TCP NewReno	80	0.098	107.97	89.38	137.81
TCP NewReno+ Mod		0.101	177.08	90.30	137.81



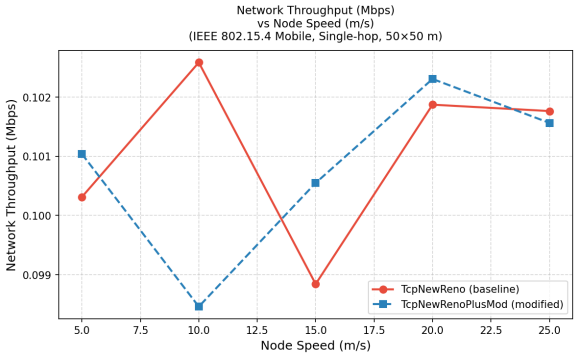
(a) Throughput vs. #nodes.



(b) Throughput vs. #flows.

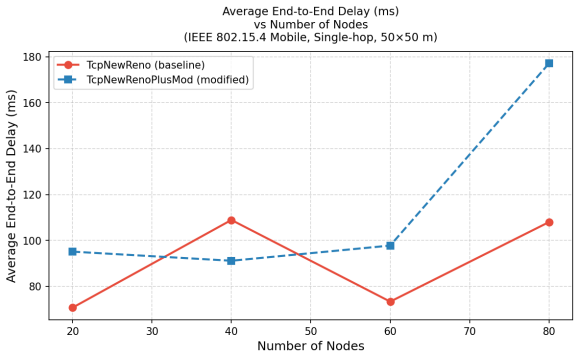


(c) Throughput vs. packet rate.

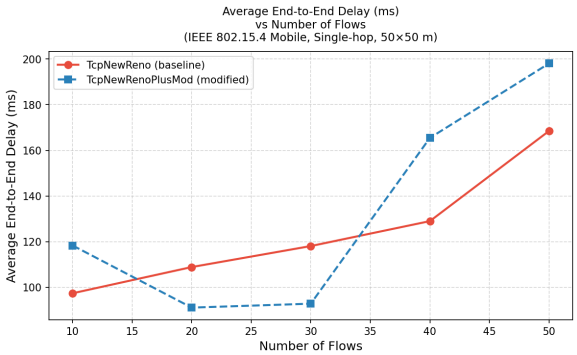


(d) Throughput vs. node speed.

Figure 14: WPAN 802.15.4 — throughput across all parameter sweeps. Values hover tightly around 0.10 Mbps throughout due to the hard 250 kbps raw link rate cap of the protocol. Neither algorithm shows a consistent advantage, as congestion-control enhancements are effectively inactive when the theoretical bandwidth ceiling is so low.

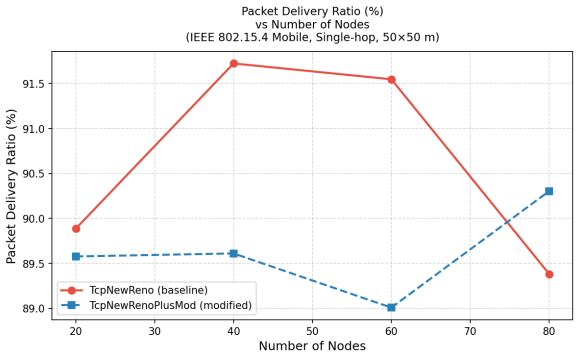


(a) Avg delay vs. #nodes.

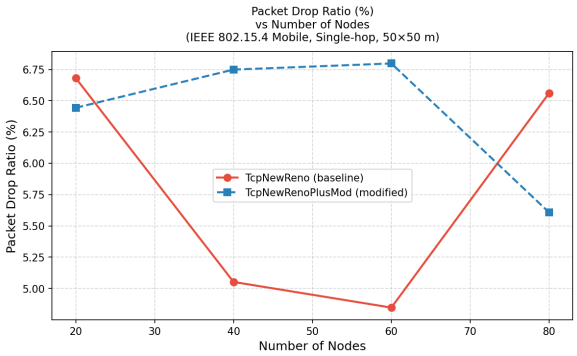


(b) Avg delay vs. #flows.

Figure 15: WPAN 802.15.4 — average end-to-end delay. The modified algorithm exhibits higher delay in several configurations: its more aggressive transmission window actively tries to keep more packets in flight over the narrow 250 kbps channel, increasing queue wait times without a proportional gain in throughput.



(a) PDR vs. #nodes.



(b) Drop ratio vs. #nodes.

Figure 16: WPAN 802.15.4 — PDR and drop ratio. Values remain overwhelmingly comparable between the two algorithms; any slight deviation is well within anticipated simulation noise boundaries.

9. Conclusion

This project successfully implemented and evaluated TCP NewReno$^+$ and our proposed extension, TCP NewReno$^+$ Modified, in the ns-3 simulation environment. By replacing the numerically unstable Timeout-to-Dupack Ratio (TDR) metric with a reliable Congestion Proportion (CP) metric, and incorporating Normalised Queueing Delay (NQD) parsed from real-time RTT measurements, the modified algorithm constructs a highly robust, three-factor loss classification mechanism.

Extensive simulation results demonstrate that the modified algorithm substantially improves upon the baseline TCP NewReno in mixed wired–wireless networks, achieving an average throughput gain of **+6.6%** (outperforming the original paper algorithm's +2.4% gain). Critically, as the wireless bit error rate increases, the performance divergence between standard TCP NewReno and TCP NewReno$^+$ Modified becomes increasingly pronounced. This is because standard TCP incorrectly interprets the heightened frequency of bit errors as severe congestion—needlessly halving its transmission window—whereas the modified algorithm accurately isolates these as wireless losses and safely bypasses the aggressive reduction. While these throughput benefits are most visible in moderate-bandwidth topologies like Wi-Fi (especially at lower node densities), the modified approach remains dynamically safe and imposes no detrimental energy overhead across the full spectrum of evaluated mobile environments.